Flow Cheatsheet — Auto-Approve / Human-Review

ApprovalFlow · End-to-end reference: submission → intake → decision (policy engine + agent + router) → payment or approval → final status

1 · Submission → Intake

```
Submitter → POST /api/invoices (JSON invoice)
```

1. **Pydantic validates** the payload against `InvoiceSubmission` — are all fields structurally correct? (schema only, no policy yet)
2. **Correlation id resolved** — reused if the caller sent one, otherwise minted fresh. *(This id traces the invoice everywhere downstream — it becomes the tracking id.)*
3. Handed to `IntakeService.submit()`:
   - **Idempotency-Key check** (if header present) — protects against the *same HTTP call* being retried.
   - **Fingerprint check** — `vendor + invoiceNumber + total` (lower-cased). Protects against the *same real-world invoice* being resubmitted. → duplicate detected here, before anything else runs.
4. **Fresh submission → two calls, in order:**
   1. Write `SubmissionRecord(status=RECEIVED)` → Redis (Dapr state API)
   2. Publish `item.submitted` → RabbitMQ (Dapr pub/sub)
5. **Only after both succeed** → HTTP handler builds the `202` response.

2 · Decision consumes `item.submitted`

Runs **two checks, sequentially** — policy engine finishes completely *before* the agent is even called (not concurrent).

① Policy engine (deterministic, no LLM)

Runs **every** rule, doesn't stop at the first hit — collects all findings, each tagged `HARD_STOP` / `HIGH` / `INFO`.

Global rules (any category)	Category rules (one block runs)
`GLOBAL-MATH` — line items + tax must reconcile to total	**MEALS** — `MEAL-03` alcohol→reject, `MEAL-01` attendee count + $75/attendee cap, `MEAL-02` client entertainment >$500 needs justification
`GLOBAL-RECEIPT` — required over $25	**SAAS** — `SAAS-01` $200/month cap
`GLOBAL-VENDOR` — unknown vendor → always human	**HARDWARE** — `HW-02` over $1,000 → always human (≤$1,000 = `HW-01`-eligible)
`GLOBAL-FX-UNKNOWN` — unsupported currency	**TRAVEL** — `TRAVEL-03` first/business class → always human, `TRAVEL-02` over $1,500 → manager approval
`GLOBAL-FX` — FX item over $1,000 or over ceiling once converted	
`GLOBAL-FRAUD` — unknown vendor + round number + (no detail or off-hours)	

② Agent (LLM)

Two tools: `fetch_policy`, `lookup_vendor`. Returns: `category`, `risk`, `confidence`, `recommendation`, `cited_rules`, `policy_violations`, `fraud_signals`.

3 · The Router

Runs in this exact order:

1. **Already processed?** → `DUPLICATE` *(dead code in production — real dupes are caught in Step 1, not here)*
2. **Any rejection** in the policy findings (e.g. `MEAL-03`)? → `REJECT`, unconditionally, any amount.
3. Otherwise, collect into one **blockers list**: hard stops + category non-compliance + over-ceiling + low-confidence.
   - Of everything on the agent's assessment, **only `confidence` is ever read** — and only to *downgrade*, never to grant approval.
4. **Blockers non-empty** → `HUMAN_REVIEW` (citing every reason at once) • **Blockers empty** → `AUTO_APPROVE`

4 · `decide()` builds the record, workflow publishes the outcome

`route → status`: `AUTO_APPROVE→AUTO_APPROVED`, `HUMAN_REVIEW→PENDING_APPROVAL`, `REJECT→REJECTED`, `DUPLICATE→DUPLICATE`.

`publish_outcome_activity` always does two things:

- Saves the `DecisionRecord` (audit trail)
- Publishes `decision.made` (→ reporting only) **and** `item.status-changed` (→ intake updates its record; notification stays quiet unless the status is terminal)

Then it branches on the route:

If `AUTO_APPROVE`	If `HUMAN_REVIEW`
3rd publish: `payment.requested`	**3rd publish:** `escalation.requested` (carries agent's `recommendation`, `confidence`, `risk`, `cited_rules` + submitter, department, vendor, usd_amount)
Orchestrator checks `route == HUMAN_REVIEW` → false → **returns immediately**, workflow done	Orchestrator checks `route == HUMAN_REVIEW` → true → **suspends** on `wait_for_external_event`, durably persisted in Redis (survives a restart)

5 · Approval service (only reached via `HUMAN_REVIEW`)

Consumes `escalation.requested` → item lands in the approvers' queue. An approver submits `approve` / `reject` / `send_back`:

1. **Claim** — atomically records the human's response (who wins if two click at once)
2. **Lease** — decides *who gets to act* on that claimed response (so concurrent retries don't duplicate the errands)
3. **Side effects**, run once, in order:
   - `raise_human_decision` — Dapr service invocation → resumes the suspended workflow
   - `publish_resolved` → `escalation.resolved`
4. **Mark done** — only after both side effects succeed. Nothing publishes after this point.

6 · Workflow resumes → final status

`apply_human_decision_activity` maps the action → status: `approve→APPROVED`, `reject→REJECTED`, `send_back→SENT_BACK`. Publishes `item.status-changed` with the final status.

Only if `approve` → also publishes `payment.requested` — joins the *exact same* downstream pipeline an auto-approved item used.

Terminal statuses (`paid`, `payment_failed`, `rejected`, `sent_back`) are the only ones notification ever acts on — everything else (`received`, `pending_approval`, `auto_approved`, `paying`) is intermediate, something is still coming next.

> **THE ONE THESIS TO RECITE COLD**
> The router reads exactly one field off the agent's output — `confidence` — and only to downgrade toward a human. The LLM's opinion never moves money by itself.